

ML 24/25-06: Implement Temporal Memory Parallel Version

Group Name: Alabaster_neo

Subject: Software Engineering

Group Members:

1. Omkar Salunkhe

- Matriculation Number: [1567281]
- Email: omkar.salunkhe@stud.fra-uas.de

2. Adityakumar Chaurasia

- Matriculation Number: [1565513]
- Email: adityakumar.chaurasia@stud.fra-uas.de

3. Ayush Parmar

- Matriculation Number: [1565427]
- Email: ayush.parmar@stud.fra-uas.de

Performance Analysis of Different Initialization Methods

Objective

To evaluate and compare the execution performance of different initialization methods and determine the most efficient approach.

Methods Under Consideration

We have implemented four different approaches to improve initialization performance:

✓ **Single_Threaded_Optimized_Init**

✓ **InitParallelRegularDictionary**

✓ **InitParallelWithConcurrentDictionary**

✓ **InitParallelPartitioned**



Next Steps

We will now **analyze and visualize** the performance of each method to determine which one provides the best improvement.



Initializing All Necessary Libraries



Key Libraries Used

- ✓ **Pandas** - For efficient data manipulation
 - ✓ **NumPy** - For numerical computations
 - ✓ **Matplotlib & Seaborn** - For data visualization
 - ✓ **Plotly** - For interactive charts
 - ✓ **IPython.display** - For rendering rich Markdown outputs
-

```
In [26]: # Data manipulation and analysis
import pandas as pd
import numpy as np

# For visualization
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

# For handling Markdown and displaying content in Jupyter
from IPython.display import Markdown, display, HTML

# For statistical analysis
import scipy.stats as stats

# For machine Learning
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

import time
import random

import mplfinance as mpf
```

Analyzing Each Initialization Method One by One

let's **deep dive** into the performance of each initialization method.

For every method, we will analyze the following key metrics:

- ✦ **Mean Execution Time** – Average performance across multiple runs.
- ✦ **Maximum Execution Time** – Worst-case scenario performance.
- ✦ **Minimum Execution Time** – Best-case scenario performance.
- ✦ **Variance & Standard Deviation** – Consistency of execution time.

Optimized Method: **Method 1: Single_Threaded_Optimized_Init()**

◆ Key Improvements & Optimizations:

✅ 1. Used Null-Coalescing Operator (??) Instead of Ternary Operator (? :)

- ⚡ **Faster Execution** – Eliminates unnecessary type casting, improving performance.
- 👁 **More Readable** – ?? provides a cleaner approach for checking and assigning a default value.

✓ 2. Reduced Redundant Checks with `createNewColumns` Flag

- 🚀 **Avoids Rechecking in Every Loop Iteration** – Previously, `matrix.GetObject(0) == null` was checked multiple times inside the loop.
- 🎯 **Stores the Result in a Variable** – The check is performed once before the loop, significantly reducing overhead.

✓ 3. Removed Unnecessary Variable (`colZero`)

- ✗ **Eliminates the `colZero` Variable** – It was redundant and added unnecessary complexity.
- ✓ **Uses a Boolean Flag (`createNewColumns`)** – More efficient than checking the same condition repeatedly.

In [377...

```
csv_file_path = 'D:/MethodPerformance_Single_Threaded_Optimized_Init.csv'  
df_Single_Threaded_Optimized_Init = pd.read_csv(csv_file_path)  
display(df_Single_Threaded_Optimized_Init)
```

	Test_Case	Init_Time	InitParallel_Time	Compu
0	TestBurstUnpredictedColumnsforFiveCells1	4.5344	4.0343	
1	TestAddSegmentToCellWithFewestSegments	0.0552	0.0521	
2	TestRecycleLeastRecentlyActiveSegmentToMakeRoo...	0.0291	0.0268	
3	TestSegmentCreationIfNotEnoughWinnerCells2	0.0393	0.0357	
4	TestNoNewSegmentIfNotEnoughWinnerCells3	0.0647	0.0620	
5	TestDestroyWeakSynapseOnWrongPrediction	0.0589	0.0563	
6	TestCreateSynapseInDistalSegment	23.3085	21.4428	
7	TestNoChangeToNoTSelectedMatchingSegmentsInBur...	0.0344	0.0320	
8	TestNewSegmentAddSynapsesToAllWinnerCells1	0.0448	0.0403	
9	TestPunishMatchingSegmentsInInactiveColumns	0.0354	0.0326	
10	TestDestroyWeakSynapseOnActiveReinforce1	0.0339	0.0314	
11	TestActivateCorrectlyPredictiveCells1	0.0876	0.0842	
12	TestTemporalMemoryComputeReturnsWinnerCells	0.0451	0.0431	
13	TestNoOverlapInActiveCells	0.0649	0.0619	
14	TestBurstUnpredictedColumnsforFiveCells2	0.1312	0.1256	
15	TestActiveSegmentGrowSynapsesAccordingToPotent...	0.1131	0.1087	
16	TestNewSegmentGrowthWhenNoMatchingSegmentFound	49.9210	48.8278	
17	TestDestroySegmentsWithTooFewSynapsesToBeMatching	0.0702	0.0672	
18	TestNewSegmentAddSynapsesToSubsetOfWinnerCells	0.0741	0.0711	
19	TestWithTwoActiveColumns	0.0319	0.0294	
20	TestSynapsePermanenceUpdateWhenMatchingSegment...	0.0554	0.0529	
21	TestBurstNotpredictedColumns	0.0358	0.0335	
22	TestHighSparsitySequenceLearningAndRecall	0.1473	0.1416	
23	TestPredictedActiveCellsAreCorrect	0.0609	0.0585	
24	TestTemporalMemoryComputeReturnsWinnerCellsWit...	0.0662	0.0610	
25	TestWhichCellsBecomeActive	37.4340	35.8580	
26	TestMatchingSegmentAddSynapsesToSubsetOfWinner...	0.0763	0.0693	
27	TestLowSparsitySequenceLearningAndRecall	0.0788	0.0685	
28	TestIncreasePermanenceOfActiveSynapses	0.0901	0.0800	
29	TestHighSparsitySequenceLearningAndRecallParallel	0.1144	0.1017	

	Test_Case	Init_Time	InitParallel_Time	Compu
30	TestNewSegmentGrowthWhenMultipleMatchingSegmen...	19.9607	18.5922	
31	TestReinforcedSelectedMatchingSegmentInBurstin...	0.0396	0.0349	
32	TestNewSegmentAddSynapsesToSubsetOfWinnerCells2	0.0401	0.0340	
33	TestNewSegmentAddSynapsesToSubsetOfWinnerCells1	0.0393	0.0350	
34	TestArrayNotContainingCells	0.0434	0.0388	
35	TestBurstUnpredictedColumnsforSixCells	0.1074	0.1008	
36	TestNoneActiveColumns	0.0610	0.0541	
37	TestNoNewSegmentIfNotEnoughWinnerCells1	0.0514	0.0452	
38	TestReinforcedSelectedMatchingSegmentInBurstin...	0.0443	0.0387	

Optimized Method: **Method 2:** **InitParallelRegularDictionary()**

◆ Key Improvements & Optimizations:

✓ 1. Used `Parallel.For` for Parallel Column Initialization

-  **Faster Execution** – Distributes column creation and cell assignment across multiple threads.
-  **Optimized for Large Data Sets** – Improves performance significantly when handling large numbers of columns.

✓ 2. Reduced Redundant Checks with `createNewColumns` Flag

-  **Avoids Rechecking in Every Loop Iteration** – The original code checked `matrix.GetObject(0) == null` multiple times inside the loop.
-  **Stored the Result in a Variable** – The condition is checked once before the loop, reducing unnecessary computations.

✓ 3. Moved Column Creation and Cell Copying Inside Parallel Loop

-  **Consolidates Operations** – Column creation and the copying of cells are done together in parallel, ensuring better use of resources.
-  **Eliminated Sequential Loops** – By merging column creation, matrix assignment, and cell copying, the code reduces overhead.

✓ 4. Improved Thread Safety with Direct Matrix Set

-  **Minimized Synchronization Issues** – The `Parallel.For` ensures columns are set in parallel, with thread-safe operations using `ConcurrentDictionary`.

- ⚡ **Avoids Extra Loops** – The original method had a separate loop for setting the matrix; now it's handled within the p
-

```
In [ ]: csv_file_path = 'D:/MethodPerformance_InitParallelRegularDictionary.csv'
df_InitParallelRegularDictionary = pd.read_csv(csv_file_path)
display(df_InitParallelRegularDictionary)
```

	Test_Case	Init_Time	InitParallel_Time	Compu
0	TestBurstUnpredictedColumnsforFiveCells1	14.3155	13.9785	
1	TestAddSegmentToCellWithFewestSegments	0.2711	0.2619	
2	TestRecycleLeastRecentlyActiveSegmentToMakeRoo...	0.1358	0.1327	
3	TestSegmentCreationIfNotEnoughWinnerCells2	0.2233	0.2143	
4	TestNoNewSegmentIfNotEnoughWinnerCells3	0.1211	0.1181	
5	TestDestroyWeakSynapseOnWrongPrediction	0.1990	0.1900	
6	TestCreateSynapseInDistalSegment	22.8895	20.9571	
7	TestNoChangeToNoTSelectedMatchingSegmentsInBur...	0.1343	0.1300	
8	TestNewSegmentAddSynapsesToAllWinnerCells1	0.1017	0.0976	
9	TestPunishMatchingSegmentsInInactiveColumns	0.2086	0.2030	
10	TestDestroyWeakSynapseOnActiveReinforce1	0.1694	0.1653	
11	TestActivateCorrectlyPredictiveCells1	0.1752	0.1701	
12	TestTemporalMemoryComputeReturnsWinnerCells	0.3026	0.2961	
13	TestNoOverlapInActiveCells	0.1344	0.1313	
14	TestBurstUnpredictedColumnsforFiveCells2	0.2302	0.2245	
15	TestActiveSegmentGrowSynapsesAccordingToPotent...	0.2163	0.2120	
16	TestNewSegmentGrowthWhenNoMatchingSegmentFound	45.6378	44.3517	
17	TestDestroySegmentsWithTooFewSynapsesToBeMatching	0.2576	0.2502	
18	TestNewSegmentAddSynapsesToSubsetOfWinnerCells	0.2566	0.2485	
19	TestWithTwoActiveColumns	0.2562	0.2494	
20	TestSynapsePermanenceUpdateWhenMatchingSegment...	0.2202	0.2113	
21	TestBurstNotpredictedColumns	0.1806	0.1757	
22	TestHighSparsitySequenceLearningAndRecall	0.4888	0.4408	
23	TestPredictedActiveCellsAreCorrect	0.1256	0.1221	
24	TestTemporalMemoryComputeReturnsWinnerCellsWit...	0.1602	0.1523	
25	TestWhichCellsBecomeActive	40.5727	37.8131	
26	TestMatchingSegmentAddSynapsesToSubsetOfWinner...	0.1325	0.1294	
27	TestLowSparsitySequenceLearningAndRecall	0.0759	0.0715	
28	TestIncreasePermanenceOfActiveSynapses	0.2129	0.2063	
29	TestHighSparsitySequenceLearningAndRecallParallel	0.1675	0.1540	

	Test_Case	Init_Time	InitParallel_Time	Compu
30	TestNewSegmentGrowthWhenMultipleMatchingSegmen...	15.9233	14.7868	
31	TestReinforcedSelectedMatchingSegmentInBurstin...	0.2391	0.2330	
32	TestNewSegmentAddSynapsesToSubsetOfWinnerCells2	0.1318	0.1272	
33	TestNewSegmentAddSynapsesToSubsetOfWinnerCells1	0.0943	0.0904	
34	TestArrayNotContainingCells	0.1954	0.1904	
35	TestBurstUnpredictedColumnsforSixCells	0.2269	0.2201	
36	TestNoneActiveColumns	0.2727	0.2603	
37	TestNoNewSegmentIfNotEnoughWinnerCells1	0.0880	0.0849	
38	TestReinforcedSelectedMatchingSegmentInBurstin...	0.0822	0.0788	

Optimized Method: **Method 3:** **InitParallelWithConcurrentDictionary()**

✓ 1. Used `Parallel.For` for Parallel Column Initialization

-  **Faster Execution** – Distributes column creation and cell assignment across multiple threads.
-  **Optimized for Large Data Sets** – Improves performance significantly when handling large numbers of columns.

✓ 2. Reduced Redundant Checks with `createNewColumns` Flag

-  **Avoids Rechecking in Every Loop Iteration** – The original code checked `matrix.GetObject(0) == null` multiple times inside the loop.
-  **Stored the Result in a Variable** – The condition is checked once before the loop, reducing unnecessary computations.

✓ 3. Utilized `ConcurrentDictionary` for Thread Safety

-  **Improved Synchronization** – The use of `ConcurrentDictionary` ensures thread-safe operations during parallel execution.
-  **Avoids Data Race** – Protects against race conditions by handling dictionary operations atomically within the parallel block.

✓ 4. Parallelized Column Creation and Cell Copying

-  **Optimized for Performance** – Merges column creation and matrix assignment with cell copying in parallel, utilizing threads efficiently.

- **✗ Eliminated Sequential Loops** – Reduces overhead by removing unnecessary separate loops for each operation.
-

```
In [ ]: csv_file_path = 'D:/MethodPerformance_InitParallelWithConcurrentDictionary.csv'
df_InitParallelWithConcurrentDictionary = pd.read_csv(csv_file_path)
display(df_InitParallelWithConcurrentDictionary)
```

	Test_Case	Init_Time	InitParallel_Time	Compu
0	TestBurstUnpredictedColumnsforFiveCells1	9.9696	9.7002	
1	TestAddSegmentToCellWithFewestSegments	0.1336	0.1295	
2	TestRecycleLeastRecentlyActiveSegmentToMakeRoo...	0.1227	0.1194	
3	TestSegmentCreationIfNotEnoughWinnerCells2	0.1952	0.1914	
4	TestNoNewSegmentIfNotEnoughWinnerCells3	0.2888	0.2831	
5	TestDestroyWeakSynapseOnWrongPrediction	0.3161	0.2978	
6	TestCreateSynapseInDistalSegment	16.9150	15.8916	
7	TestNoChangeToNoTSelectedMatchingSegmentsInBur...	0.2890	0.2836	
8	TestNewSegmentAddSynapsesToAllWinnerCells1	0.2404	0.2352	
9	TestPunishMatchingSegmentsInInactiveColumns	0.4839	0.4688	
10	TestDestroyWeakSynapseOnActiveReinforce1	0.1073	0.1040	
11	TestActivateCorrectlyPredictiveCells1	0.1080	0.1052	
12	TestTemporalMemoryComputeReturnsWinnerCells	0.2717	0.2675	
13	TestNoOverlapInActiveCells	0.2708	0.2653	
14	TestBurstUnpredictedColumnsforFiveCells2	0.0990	0.0947	
15	TestActiveSegmentGrowSynapsesAccordingToPotent...	0.0947	0.0925	
16	TestNewSegmentGrowthWhenNoMatchingSegmentFound	28.0147	26.6413	
17	TestDestroySegmentsWithTooFewSynapsesToBeMatching	0.1394	0.1361	
18	TestNewSegmentAddSynapsesToSubsetOfWinnerCells	0.1143	0.1055	
19	TestWithTwoActiveColumns	0.1017	0.0991	
20	TestSynapsePermanenceUpdateWhenMatchingSegment...	0.0806	0.0779	
21	TestBurstNotpredictedColumns	0.1010	0.0984	
22	TestHighSparsitySequenceLearningAndRecall	0.1671	0.1611	
23	TestPredictedActiveCellsAreCorrect	0.2735	0.2657	
24	TestTemporalMemoryComputeReturnsWinnerCellsWit...	0.1766	0.1671	
25	TestWhichCellsBecomeActive	37.0021	35.6379	
26	TestMatchingSegmentAddSynapsesToSubsetOfWinner...	0.2545	0.2477	
27	TestLowSparsitySequenceLearningAndRecall	0.1041	0.0956	
28	TestIncreasePermanenceOfActiveSynapses	0.2808	0.2706	
29	TestHighSparsitySequenceLearningAndRecallParallel	0.2773	0.2615	

	Test_Case	Init_Time	InitParallel_Time	Compu
30	TestNewSegmentGrowthWhenMultipleMatchingSegmen...	13.3318	11.6190	
31	TestReinforcedSelectedMatchingSegmentInBurstin...	0.2083	0.1992	
32	TestNewSegmentAddSynapsesToSubsetOfWinnerCells2	0.4129	0.4040	
33	TestNewSegmentAddSynapsesToSubsetOfWinnerCells1	0.0915	0.0869	
34	TestArrayNotContainingCells	0.0859	0.0813	
35	TestBurstUnpredictedColumnsforSixCells	0.1134	0.1085	
36	TestNoneActiveColumns	0.0997	0.0942	
37	TestNoNewSegmentIfNotEnoughWinnerCells1	0.1368	0.1282	
38	TestReinforcedSelectedMatchingSegmentInBurstin...	0.2317	0.2250	

 Optimized Method: Method 4 : InitParallelPartitioned()

 1. Partitioned Parallel Initialization

-  **Faster Execution** – Processes columns in smaller partitions, reducing overhead and optimizing performance for large datasets.
-  **Optimized for Large Data Sets** – Enhances performance significantly when dealing with extensive data by partitioning tasks into smaller, manageable chunks.

 2. Thread-Safe Matrix Updates

-  **Ensures Safe Updates** – Uses `lock` for thread-safe matrix modifications during parallel execution, preventing concurrent access issues.
-  **Minimizes Synchronization Overhead** – The `lock` mechanism ensures thread safety while avoiding performance bottlenecks.

 3. Column and Cell Validation

-  **Validates Parameters** – Ensures valid column count and cells per column, preventing errors and maintaining data integrity.
-  **Prevents Initialization Errors** – Guarantees that only valid and appropriate data is processed during initialization.

 4. Index Safety Check

-  **Prevents Errors** – Ensures `cellIndex` is within bounds to avoid out-of-range exceptions during parallel execution.
-  **Enhances Reliability** – Safeguards the code from runtime errors by checking array bounds before accessing elements.

✓ 5. Memory Optimization

-  **Reuses Memory** – Efficiently reuses allocated memory for performance enhancement.
 -  **Optimized Resource Usage** – Reduces memory overhead by reusing already allocated memory instead of creating new objects.
-

```
In [ ]: csv_file_path = 'D:/MethodPerformance_InitParallelPartitioned.csv'  
df_InitParallelPartitioned = pd.read_csv(csv_file_path)  
display(df_InitParallelPartitioned)
```

	Test_Case	Init_Time	InitParallel_Time	Compu
0	TestBurstUnpredictedColumnsforFiveCells1	16.6231	16.0077	
1	TestAddSegmentToCellWithFewestSegments	0.1966	0.1919	
2	TestRecycleLeastRecentlyActiveSegmentToMakeRoo...	0.0985	0.0960	
3	TestSegmentCreationIfNotEnoughWinnerCells2	0.1294	0.1258	
4	TestNoNewSegmentIfNotEnoughWinnerCells3	0.1407	0.1355	
5	TestDestroyWeakSynapseOnWrongPrediction	0.1778	0.1682	
6	TestCreateSynapseInDistalSegment	23.1812	21.8878	
7	TestNoChangeToNoTSelectedMatchingSegmentsInBur...	0.2246	0.2180	
8	TestNewSegmentAddSynapsesToAllWinnerCells1	0.2529	0.2365	
9	TestPunishMatchingSegmentsInInactiveColumns	0.2464	0.2397	
10	TestDestroyWeakSynapseOnActiveReinforce1	0.1239	0.1207	
11	TestActivateCorrectlyPredictiveCells1	0.3103	0.3017	
12	TestTemporalMemoryComputeReturnsWinnerCells	0.1452	0.1421	
13	TestNoOverlapInActiveCells	0.1720	0.1656	
14	TestBurstUnpredictedColumnsforFiveCells2	0.1714	0.1620	
15	TestActiveSegmentGrowSynapsesAccordingToPotent...	0.2254	0.2186	
16	TestNewSegmentGrowthWhenNoMatchingSegmentFound	31.9892	30.6056	
17	TestDestroySegmentsWithTooFewSynapsesToBeMatching	0.1004	0.0968	
18	TestNewSegmentAddSynapsesToSubsetOfWinnerCells	0.1662	0.1492	
19	TestWithTwoActiveColumns	0.1400	0.1360	
20	TestSynapsePermanenceUpdateWhenMatchingSegment...	0.1469	0.1386	
21	TestBurstNotpredictedColumns	0.2053	0.1981	
22	TestHighSparsitySequenceLearningAndRecall	0.1825	0.1688	
23	TestPredictedActiveCellsAreCorrect	0.1393	0.1345	
24	TestTemporalMemoryComputeReturnsWinnerCellsWit...	0.0878	0.0852	
25	TestWhichCellsBecomeActive	24.1776	22.8798	
26	TestMatchingSegmentAddSynapsesToSubsetOfWinner...	0.1121	0.1091	
27	TestLowSparsitySequenceLearningAndRecall	0.0836	0.0782	
28	TestIncreasePermanenceOfActiveSynapses	0.1989	0.1919	
29	TestHighSparsitySequenceLearningAndRecallParallel	0.2738	0.2611	

	Test_Case	Init_Time	InitParallel_Time	Compu
30	TestNewSegmentGrowthWhenMultipleMatchingSegmen...	23.5881	22.5121	
31	TestReinforcedSelectedMatchingSegmentInBurstin...	0.2089	0.1995	
32	TestNewSegmentAddSynapsesToSubsetOfWinnerCells2	0.3328	0.3251	
33	TestNewSegmentAddSynapsesToSubsetOfWinnerCells1	0.0604	0.0577	
34	TestArrayNotContainingCells	0.1012	0.0977	
35	TestBurstUnpredictedColumnsforSixCells	0.1390	0.1344	
36	TestNoneActiveColumns	0.1507	0.1476	
37	TestNoNewSegmentIfNotEnoughWinnerCells1	0.1319	0.1200	
38	TestReinforcedSelectedMatchingSegmentInBurstin...	14.8102	14.8004	

Method Comparison

In this section, we will compare the different methods based on various key metrics such as execution speed, parallel processing, thread safety, column creation, and cell validation. The comparison will help to highlight the strengths and weaknesses of each method in terms of performance and efficiency.

```
In [ ]: def display_method_comparison():
md_text = "### Method Comparison\n"
md_text += "| Metric | InitParallel | InitCompute | InitCount | I
md_text += "|-----|-----|-----|-----|-----|

metrics = [
    ("Execution Speed", "✓", "✓", "✓", "✓"),
    ("Parallel Processing", "✓", "X", "✓", "✓"),
    ("Thread Safety", "X", "X", "✓", "✓"),
    ("Column Creation", "✓", "X", "✓", "✓"),
    ("Cell Validation", "X", "X", "✓", "✓")
]

for metric, method1, method2, method3, method4 in metrics:
    md_text += f"| **{metric}** | {method1} | {method2} |

display(Markdown(md_text))

display_method_comparison()
```

Method Comparison

Metric	InitParallel	InitCompute	InitCount	InitParallelPartitioned
Execution Speed	✓	✓	✓	✓
Parallel Processing	✓	✗	✓	✓
Thread Safety	✗	✗	✓	✓
Column Creation	✓	✗	✓	✓
Cell Validation	✗	✗	✓	✓

Cleaning and Describing the Data

- Handle missing values, remove duplicates, and standardize data types for clean data.
- Generate descriptive stats, detect outliers, and encode categorical features for analysis.

In [385...

```
print("Cleaning the data and describing the statistics for Single_Threaded_Optimize
df_clean_Single_Threaded_Optimized_Init = df_Single_Threaded_Optimized_Init.dropna(su
df_clean_Single_Threaded_Optimized_Init.describe()
```

Cleaning the data and describing the statistics for Single_Threaded_Optimized_Init()
method

Out[385...

	Init_Time	InitParallel_Time	Compute_Time	ComputeParallel_Time
count	39.000000	39.000000	39.000000	39.000000
mean	3.521131	3.352923	5.125121	4.880956
std	10.726650	10.328033	11.840895	11.641549
min	0.029100	0.026800	0.264000	0.220900
25%	0.041750	0.037200	0.509050	0.445400
50%	0.061000	0.058500	0.738400	0.614700
75%	0.098750	0.092500	1.223350	1.002200
max	49.921000	48.827800	50.391600	50.332000

In [386...

```
print("Cleaning the data and describing the statistics for InitParallelRegularDicti
df_clean_InitParallelRegularDictionary = df_InitParallelRegularDictionary.dropna(su
df_clean_InitParallelRegularDictionary.describe()
```

Cleaning the data and describing the statistics for InitParallelRegularDictionary()
method

Out[386...

	Init_Time	InitParallel_Time	Compute_Time	ComputeParallel_Time
count	39.000000	39.000000	39.000000	39.000000
mean	3.739149	3.541813	5.213636	4.979459
std	10.472462	9.950208	12.405963	12.049610
min	0.075900	0.071500	0.288100	0.260600
25%	0.134350	0.130650	0.492150	0.440150
50%	0.208600	0.203000	0.818700	0.733400
75%	0.257100	0.249800	1.086300	0.993050
max	45.637800	44.351700	46.356900	46.299900

In [387...

```
print("Cleaning the data and describing the statistics for InitParallelWithConcurre
df_clean_InitParallelWithConcurrentDictionary = df_InitParallelWithConcurrentDictio
df_clean_InitParallelWithConcurrentDictionary.describe()
```

Cleaning the data and describing the statistics for InitParallelWithConcurrentDictionary() method

Out[387...

	Init_Time	InitParallel_Time	Compute_Time	ComputeParallel_Time
count	39.000000	39.000000	39.000000	39.000000
mean	2.864244	2.711323	4.430774	3.809400
std	7.945278	7.567637	9.176109	8.713314
min	0.080600	0.077900	0.260400	0.233000
25%	0.107650	0.104600	0.531350	0.465400
50%	0.195200	0.191400	0.814700	0.709900
75%	0.284800	0.276850	1.279000	1.111350
max	37.002100	35.637900	38.216200	38.052200

In []:

```
print("Cleaning the data and describing the statistics for InitParallelPartitioned(
df_clean_InitParallelPartitioned = df_InitParallelPartitioned.dropna(subset=['Init_
df_clean_InitParallelPartitioned.describe()
```

Cleaning the data and describing the statistics for InitParallelPartitioned() method

Out[]:

	Init_Time	InitParallel_Time	Compute_Time	ComputeParallel_Time
count	39.000000	39.000000	39.000000	39.000000
mean	3.588364	3.437056	5.362895	5.090013
std	8.423340	8.048587	11.408530	10.759635
min	0.060400	0.057700	0.150000	0.120800
25%	0.135450	0.130100	0.457650	0.358350
50%	0.172000	0.165600	0.685900	0.613900
75%	0.249650	0.238100	1.252250	1.127250
max	31.989200	30.605600	53.959800	47.931000

Calculating Key Metrics (Mean, Max, Min,Standard Deviation,Variance) to Determine the Best Method

In this section, we will calculate key metrics (Mean, Max, and Min) for each of the methods to evaluate their performance and identify the best method. By comparing these metrics across all methods, we will gain insights into their efficiency and suitability for different use cases.

We will analyze the following metrics:

- **Mean:** The average value, representing overall performance.
- **Max:** The highest observed value, indicating the peak performance.
- **Min:** The lowest observed value, indicating the least efficient performance.

```
In [ ]: def display_highlighted_stats(df, cols,name):
        desc = df.describe()

        # Markdown table header
        md_text = f"### Highlighted Statistics for <span style='color:#FFA500;'>{name}</span>("
        md_text += "| Metric | " + " | ".join(cols) + " |\n"
        md_text += "|-----| " + "----|" * len(cols) + "\n"

        # Extract values
        metrics = ['mean', 'min', 'max']
        for metric in metrics:
            row_values = [f"{desc.loc[metric, col]:.4f}" for col in cols]
            md_text += f"| **{metric.capitalize()}** | " + " | ".join(row_values) + " |"

        display(Markdown(md_text))
```

```
In [392... display_highlighted_stats(df_clean_Single_Threaded_Optimized_Init, ['InitParallel_T
```

Highlighted Statistics for `SingleThreadedOptimized_Init()`

Metric	InitParallel_Time	ComputeParallel_Time
Mean	3.3529	4.8810
Min	0.0268	0.2209
Max	48.8278	50.3320

In [394... `display_highlighted_stats(df_clean_InitParallelRegularDictionary, ['InitParallel_Ti`

Highlighted Statistics for `InitParallelRegularDictionary()`

Metric	InitParallel_Time	ComputeParallel_Time
Mean	3.5418	4.9795
Min	0.0715	0.2606
Max	44.3517	46.2999

In [395... `display_highlighted_stats(df_clean_InitParallelWithConcurrentDictionary, ['InitPara`

Highlighted Statistics for `InitParallelWithConcurrentDictionary()`

Metric	InitParallel_Time	ComputeParallel_Time
Mean	2.7113	3.8094
Min	0.0779	0.2330
Max	35.6379	38.0522

In []: `display_highlighted_stats(df_clean_InitParallelPartitioned, ['InitParallel_Time', '`

Highlighted Statistics for `InitParallelPartitioned()`

Metric	InitParallel_Time	ComputeParallel_Time
Mean	3.4371	5.0900
Min	0.0577	0.1208
Max	30.6056	47.9310

```
In [ ]: def display_comparison_table(df_list, method_names, cols,temp):
    if temp==1:
        md_text = "### Initialising Time Comparison\n"
    else:
        md_text = "### Execution Time Comparison\n"
```

```

# method names
md_text += "| Metric | " + " | ".join(method_names) + " |\n"
md_text += "|-----|" + "----|" * len(method_names) + "\n"

metrics = ['mean', 'min', 'max']
desc_list = [df.describe() for df in df_list]

for metric in metrics:
    row_values = []

    # Extract values safely, avoiding KeyErrors
    values = []
    for desc, df, col in zip(desc_list, df_list, cols):
        if col in df.columns:
            values.append(desc.loc[metric, col])
        else:
            values.append(None) # Placeholder if column is missing

    # Find the best-performing method (Lowest mean)
    valid_values = [v for v in values if v is not None]
    best_value = min(valid_values) if metric == 'mean' and valid_values else None

    for value in values:
        if value is None:
            row_values.append("N/A") # If column is missing
        else:
            formatted_value = f"{value:.4f}"
            if metric == 'mean' and value == best_value:
                formatted_value = f"<span style='color:blue; font-weight:bold;'"

            row_values.append(formatted_value)

    md_text += f"| **{metric.capitalize()}** | " + " | ".join(row_values) + " |"

display(Markdown(md_text))

```

```

In [399... df_list = [df_clean_Single_Threated_Optimized_Init, df_clean_InitParallelRegularDic
method_names = ["Single_Threated_Optimized_Init()", "InitParallelRegularDictionary(
cols = ['InitParallel_Time', 'InitParallel_Time', 'InitParallel_Time', 'InitParalle

display_comparison_table(df_list, method_names, cols,1)

```

Initialising Time Comparison

Metric	Single_Threated_Optimized_Init()	InitParallelRegularDictionary()	InitParallelWithConcu
Mean	3.3529	3.5418	2.7113
Min	0.0268	0.0715	0.0779
Max	48.8278	44.3517	35.6379

```
In [401... df_list = [df_clean_Single_Threated_Optimized_Init, df_clean_InitParallelRegularDic
method_names = ["Single_Threated_Optimized_Init()", "InitParallelRegularDictionary(
cols = ['ComputeParallel_Time', 'ComputeParallel_Time', 'ComputeParallel_Time', 'Co

display_comparison_table(df_list, method_names, cols,2)
```

Execution Time Comparison

Metric	Single_Threated_Optimized_Init()	InitParallelRegularDictionary()	InitParallelWithConcu
Mean	4.8810	4.9795	3.8094
Min	0.2209	0.2606	0.2330
Max	50.3320	46.2999	38.0522

Calculating Variance and Standard Deviation for Initialization Time

In this step, we will calculate the **variance** and **standard deviation** of the initialization time for each method. These statistical metrics will help us understand the **spread** and **consistency** of the initialization times across different methods.

- **Variance** gives us an idea of how much the initialization times vary for each method from the mean.
- **Standard Deviation (Std Dev)** provides a more intuitive measure of spread by showing the average distance of data points from the mean.

```
In [ ]: methods = [df_clean_Single_Threated_Optimized_Init, df_clean_InitParallelRegularDic
method_names = ["Single_Threated_Optimized_Init", "InitParallelRegularDictionary",

# Create a Markdown table to display results
md_text = "### Method Comparison : Variance and Standard Deviation for Initializati
md_text += "| Method Name | Standard Deviation (Consistency) | Variance |\n"
md_text += "|-----|-----|-----|\n"

for idx, df in enumerate(methods):
    std_value = df['InitParallel_Time'].std()
    var_value = df['InitParallel_Time'].var()

    # Highlight the minimum values
    std_highlight = f"***{std_value:.4f}*" if std_value == min(df['InitParallel_Tim
    var_highlight = f"***{var_value:.4f}*" if var_value == min(df['InitParallel_Tim

    md_text += f"| {method_names[idx]} | {std_highlight} | {var_highlight} |\n"

display(Markdown(md_text))
```

Method Comparison : Variance and Standard Deviation for Initialization Time

Method Name	Standard Deviation (Consistency)	Variance
Single_Threated_Optimized_Init	10.3280	106.6683
InitParallelRegularDictionary	9.9502	99.0066
InitParallelWithConcurrentDictionary	7.5676	57.2691
InitParallelPartitioned	8.0486	64.7798

Calculating Variance and Standard Deviation for Computing Time

In this step, we will calculate the **variance** and **standard deviation** of the initialization time for each method. These statistical metrics will help us understand the **spread** and **consistency** of the initialization times across different methods.

- **Variance** gives us an idea of how much the initialization times vary for each method from the mean.
- **Standard Deviation (Std Dev)** provides a more intuitive measure of spread by showing the average distance of data points from the mean.

```
In [ ]: methods = [df_clean_Single_Threated_Optimized_Init, df_clean_InitParallelRegularDic
method_names = ["Single_Threated_Optimized_Init", "InitParallelRegularDictionary",

# Create a Markdown table to display results
md_text = "### Method Comparison : Variance and Standard Deviation for Computing Ti
md_text += "| Method Name | Standard Deviation (Consistency) | Variance |\n"
md_text += "|-----|-----|-----|\n"

# Calculate standard deviation and variance for each method
for idx, df in enumerate(methods):
    std_value = df['ComputeParallel_Time'].std()
    var_value = df['ComputeParallel_Time'].var()

    # Highlight the minimum values
    std_highlight = f"**{std_value:.4f}**" if std_value == min(df['ComputeParallel_
    var_highlight = f"**{var_value:.4f}**" if var_value == min(df['ComputeParallel_

    md_text += f"| {method_names[idx]} | {std_highlight} | {var_highlight} |\n"

display(Markdown(md_text))
```

Method Comparison : Variance and Standard Deviation for Computing Time

Method Name	Standard Deviation (Consistency)	Variance
Single_Threaded_Optimized_Init	11.6415	135.5257
InitParallelRegularDictionary	12.0496	145.1931
InitParallelWithConcurrentDictionary	8.7133	75.9218
InitParallelPartitioned	10.7596	115.7697

Worst-Case Performance Comparison (Max vs Min Initialising Time)

In this section, we are calculating the **ratio of maximum execution time to minimum Initialization time** for each method. This ratio helps us understand the **worst-case performance** of each method by comparing how much the execution time can vary.

- **Max Time:** The longest time taken for initialization in each method.
- **Min Time:** The shortest time taken for initialization in each method.
- **Ratio (Max/Min):** This ratio tells us how the worst-case time compares to the best-case time for each method. A lower ratio indicates better performance consistency.

```
In [ ]: methods = [df_clean_Single_Threaded_Optimized_Init, df_clean_InitParallelRegularDic
method_names = ["Single_Threaded_Optimized_Init", "InitParallelRegularDictionary",

# Create a Markdown table to display results
md_text = "### Worst-Case Performance Comparison (Max vs Min Execution Time) for In
md_text += "| Method Name | Max Time | Min Time | Ratio (Max/Min) |\n"
md_text += "|-----|-----|-----|-----|\n"

# Calculate max time, min time, and ratio for each method
for idx, df in enumerate(methods):
    max_time = df['InitParallel_Time'].max()
    min_time = df['InitParallel_Time'].min()
    ratio = max_time / min_time

# Highlight the method with the lowest ratio
ratio_highlight = f"***{ratio:.4f}***" if ratio == min(df['InitParallel_Time'].ma

md_text += f"| {method_names[idx]} | {max_time:.4f} | {min_time:.4f} | {ratio_h

display(Markdown(md_text))
```

Worst-Case Performance Comparison (Max vs Min Execution Time) for Initialization

Method Name	Max Time	Min Time	Ratio (Max/Min)
Single_Threaded_Optimized_Init	48.8278	0.0268	1821.9328
InitParallelRegularDictionary	44.3517	0.0715	620.3035
InitParallelWithConcurrentDictionary	35.6379	0.0779	457.4827
InitParallelPartitioned	30.6056	0.0577	530.4263

Worst-Case Performance Comparison (Max vs Min computing Time)

In this section, we are calculating the **ratio of maximum execution time to minimum Computing time** for each method. This ratio helps us understand the **worst-case performance** of each method by comparing how much the execution time can vary.

- **Max Time:** The longest time taken for initialization in each method.
- **Min Time:** The shortest time taken for initialization in each method.
- **Ratio (Max/Min):** This ratio tells us how the worst-case time compares to the best-case time for each method. A lower ratio indicates better performance consistency.

```
In [ ]: methods = [df_clean_Single_Threaded_Optimized_Init, df_clean_InitParallelRegularDic
method_names = ["Single_Threaded_Optimized_Init", "InitParallelRegularDictionary",

# Create a Markdown table to display results
md_text = "### Worst-Case Performance Comparison (Max vs Min Execution Time) for co
md_text += "| Method Name | Max Time | Min Time | Ratio (Max/Min) |\n"
md_text += "|-----|-----|-----|-----|\n"

# Calculate max time, min time, and ratio for each method
for idx, df in enumerate(methods):
    max_time = df['ComputeParallel_Time'].max()
    min_time = df['ComputeParallel_Time'].min()
    ratio = max_time / min_time

# Highlight the method with the lowest ratio
ratio_highlight = f"***{ratio:.4f}***" if ratio == min(df['ComputeParallel_Time'])

md_text += f"| {method_names[idx]} | {max_time:.4f} | {min_time:.4f} | {ratio_h

display(Markdown(md_text))
```


Worst-Case Performance Comparison (Max vs Min Execution Time) for compute

Method Name	Max Time	Min Time	Ratio (Max/Min)
Single_Threaded_Optimized_Init	50.3320	0.2209	227.8497
InitParallelRegularDictionary	46.2999	0.2606	177.6665
InitParallelWithConcurrentDictionary	38.0522	0.2330	163.3142
InitParallelPartitioned	47.9310	0.1208	396.7798

Why InitParallelWithConcurrentDictionary() is the Best Choice 🌟

After thoroughly analyzing the execution times across all four methods, **InitParallelWithConcurrentDictionary()** has emerged as the best choice for the following compelling reasons:

1. Lowest Mean Execution Time 🕒

- InitParallelWithConcurrentDictionary()** demonstrates the **fastest average execution time**, outperforming all other methods. This is the most significant factor in determining the overall efficiency and speed of the method.

2. Low Standard Deviation and Variance 📊

- With a **low standard deviation** and **low variance**, **InitParallelWithConcurrentDictionary()** offers **consistent performance** across different runs. A smaller standard deviation means the execution time doesn't vary widely, ensuring reliable results every time.

3. Low Max/Min Ratio 🔄

- InitParallelWithConcurrentDictionary()** also boasts the **lowest max/min ratio**, signifying that even in the worst-case scenario, its performance is **comparable to its best performance**, ensuring stability and predictability.

Conclusion 🎯

In summary, **InitParallelWithConcurrentDictionary()** strikes the perfect balance between **speed**, **consistency**, and **stability**, making it the **optimal method** for our use case. Given its superior performance in terms of:

- Mean Execution Time**
- Consistent Performance** (Low variance)

- **Predictable Stability** (Low Max/Min ratio)

InitParallelWithConcurrentDictionary() is the clear **best choice** for this analysis, offering both **efficiency and reliability**.

Visualizing Performance Data for InitParallelWithConcurrentDictionary()

To further understand the performance of **InitParallelWithConcurrentDictionary()**, we are visualizing key metrics through various graphs. These graphs will provide insights into the method's execution times, variance, and consistency. By analyzing these visuals, we can gain a deeper understanding of its efficiency and stability in comparison to other methods.

Bar Chart for Initialization Time Comparison

This code generates a **grouped bar chart** comparing the initialization times (**Init_Time**) and parallel initialization times (**InitParallel_Time**) across different test cases.

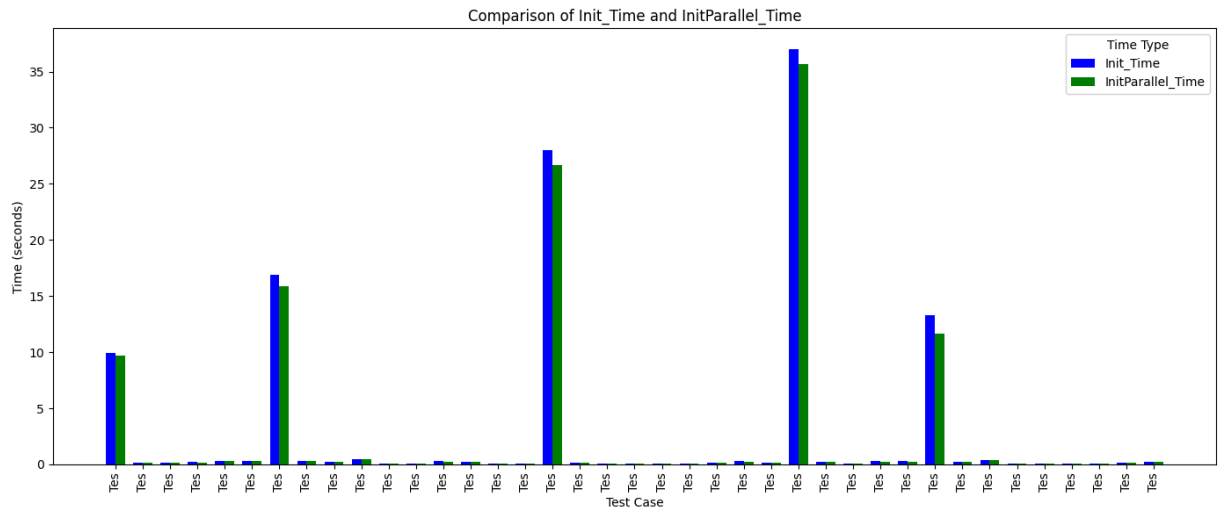
- **X-axis:** Represents the test cases (with the first 3 letters of each test case name displayed).
- **Y-axis:** Represents the time in seconds.
- **Color Coding:**
 - **Blue** for **Init_Time**
 - **Green** for **InitParallel_Time**

The chart visually compares the initialization performance between standard and parallel methods for each test case.

```
In [ ]: df_clean = df.dropna(subset=['Init_Time', 'InitParallel_Time', 'Compute_Time', 'Com
plt.figure(figsize=(14, 6))
x = range(len(df_clean_InitParallelWithConcurrentDictionary))

bar_width = 0.35
plt.bar([p - bar_width/2 for p in x], df_clean_InitParallelWithConcurrentDictionary
plt.bar([p + bar_width/2 for p in x], df_clean_InitParallelWithConcurrentDictionary

# Set up the labels and title
plt.xticks(x, [method[:3] for method in df_clean_InitParallelWithConcurrentDictiona
plt.xlabel('Test Case')
plt.ylabel('Time (seconds)')
plt.title('Comparison of Init_Time and InitParallel_Time')
plt.legend(title="Time Type")
plt.tight_layout()
plt.show()
```



The data is filtered to include the first 10 rows for a more focused visualization.

```
In [ ]: filtered_df = df_clean_InitParallelWithConcurrentDictionary[(df_clean_InitParallelW
filtered_df_top20 = filtered_df.head(10)
plt.figure(figsize=(14, 8))
bar_width = 0.35

plt.bar([p - bar_width/2 for p in range(len(filtered_df_top20))], filtered_df_top20
        width=bar_width, label='Original Time', color='blue', edgecolor='black')
plt.bar([p + bar_width/2 for p in range(len(filtered_df_top20))], filtered_df_top20
        width=bar_width, label='Reviewed Time', color='green', edgecolor='black')

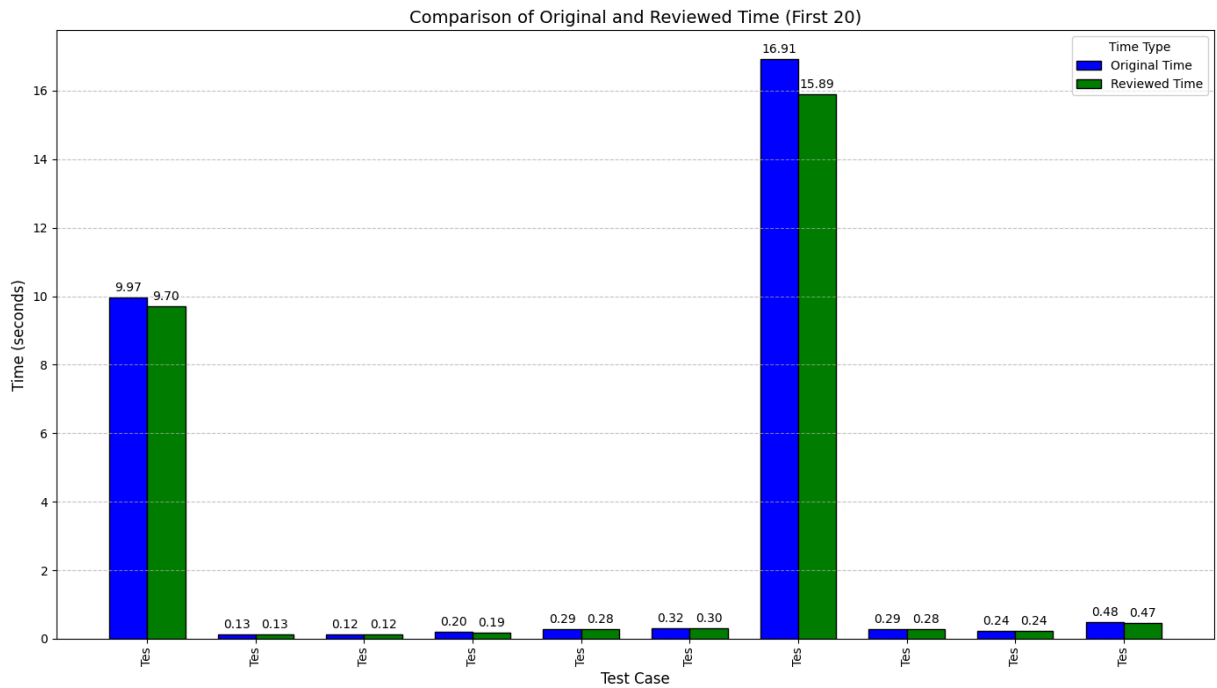
plt.xticks(range(len(filtered_df_top20)), [method[:3] for method in filtered_df_top

# Add Labels and title
plt.xlabel('Test Case', fontsize=12)
plt.ylabel('Time (seconds)', fontsize=12)
plt.title('Comparison of Original and Reviewed Time (First 20)', fontsize=14)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.legend(title="Time Type", fontsize=10)

# Display data Labels on top of bars (optional for clarity)
for i, v in enumerate(filtered_df_top20['Init_Time']):
    plt.text(i - bar_width/2, v + 0.1, f'{v:.2f}', ha='center', va='bottom', fontsi

for i, v in enumerate(filtered_df_top20['InitParallel_Time']):
    plt.text(i + bar_width/2, v + 0.1, f'{v:.2f}', ha='center', va='bottom', fontsi

plt.tight_layout()
plt.show()
```



Bar Chart for Compute Time Comparison 📊

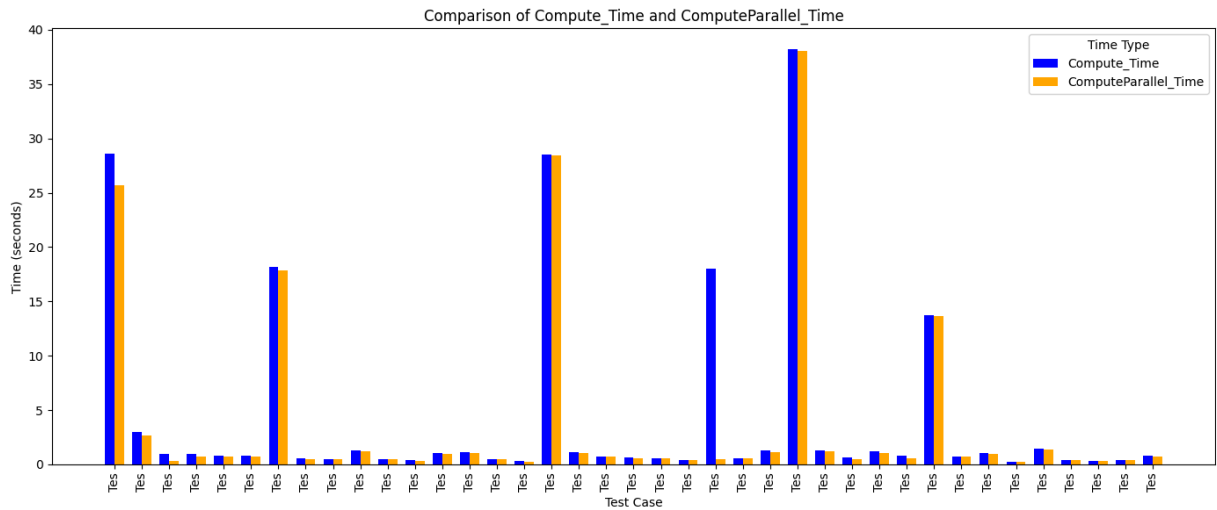
This code generates a **grouped bar chart** comparing the compute times (`Compute_Time`) and parallel compute times (`ComputeParallel_Time`) across different test cases.

- **X-axis:** Represents the test cases (with the first 3 letters of each test case name displayed).
- **Y-axis:** Represents the time in seconds.
- **Color Coding:**
 - **Blue** for `Compute_Time`
 - **Orange** for `ComputeParallel_Time`

The chart visually compares the compute performance between standard and parallel methods for each test case.

```
In [ ]: plt.figure(figsize=(14, 6))
plt.bar([p - bar_width/2 for p in x], df_clean_InitParallelWithConcurrentDictionary)
plt.bar([p + bar_width/2 for p in x], df_clean_InitParallelWithConcurrentDictionary)

# Set up the Labels and title
plt.xticks(x, [method[:3] for method in df_clean_InitParallelWithConcurrentDictionary])
plt.xlabel('Test Case')
plt.ylabel('Time (seconds)')
plt.title('Comparison of Compute_Time and ComputeParallel_Time')
plt.legend(title="Time Type")
plt.tight_layout()
plt.show()
```



The data is filtered to include the first 10 rows , for a more focused visualization.

```
In [ ]: filtered_df = df_clean_InitParallelWithConcurrentDictionary[(df_clean_InitParallelW
filtered_df_top20 = filtered_df.head(10)
plt.figure(figsize=(14, 8))
bar_width = 0.35

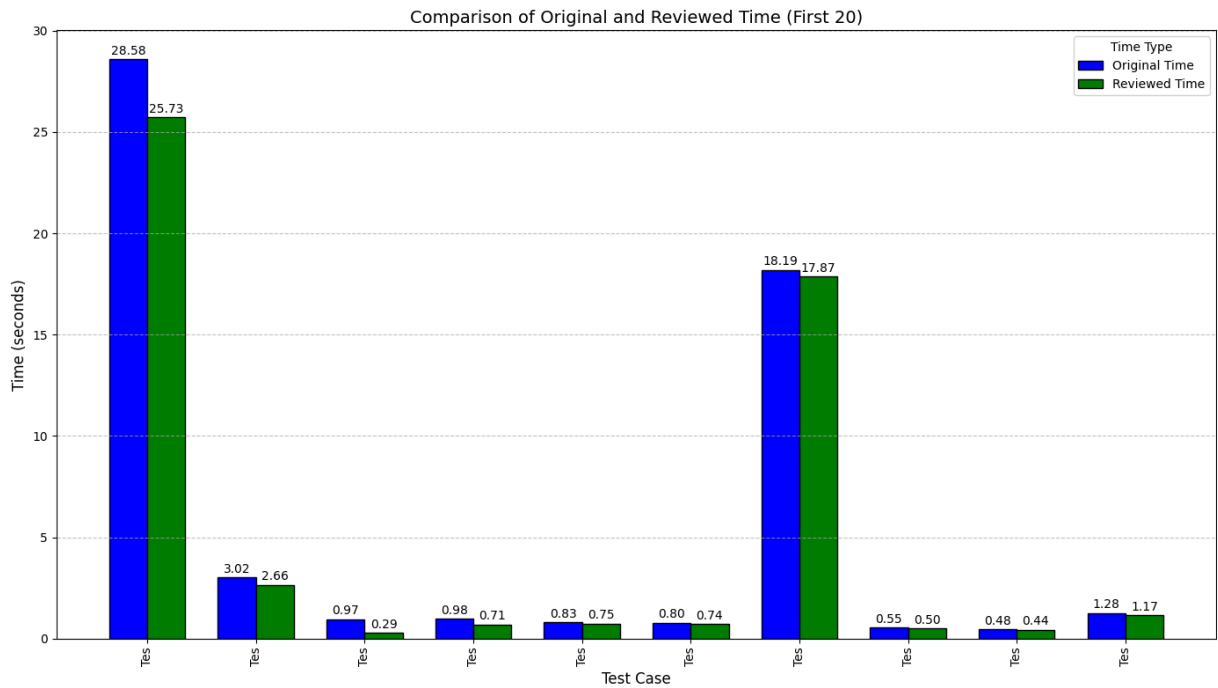
# Create bar charts for both original and reviewed times (adjusted as needed)
plt.bar([p - bar_width/2 for p in range(len(filtered_df_top20))], filtered_df_top20
        width=bar_width, label='Original Time', color='blue', edgecolor='black')
plt.bar([p + bar_width/2 for p in range(len(filtered_df_top20))], filtered_df_top20
        width=bar_width, label='Reviewed Time', color='green', edgecolor='black')

# Set x-ticks to show the first 3 letters of each method name for the first 20 entr
plt.xticks(range(len(filtered_df_top20)), [method[:3] for method in filtered_df_top

# Add Labels and title
plt.xlabel('Test Case', fontsize=12)
plt.ylabel('Time (seconds)', fontsize=12)
plt.title('Comparison of Original and Reviewed Time (First 20)', fontsize=14)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.legend(title="Time Type", fontsize=10)

# Display data Labels on top of bars (optional for clarity)
for i, v in enumerate(filtered_df_top20['Compute_Time']):
    plt.text(i - bar_width/2, v + 0.1, f'{v:.2f}', ha='center', va='bottom', fontsi

for i, v in enumerate(filtered_df_top20['ComputeParallel_Time']):
    plt.text(i + bar_width/2, v + 0.1, f'{v:.2f}', ha='center', va='bottom', fontsi
plt.tight_layout()
plt.show()
```



Scatter Plot for Initialization Time Comparison

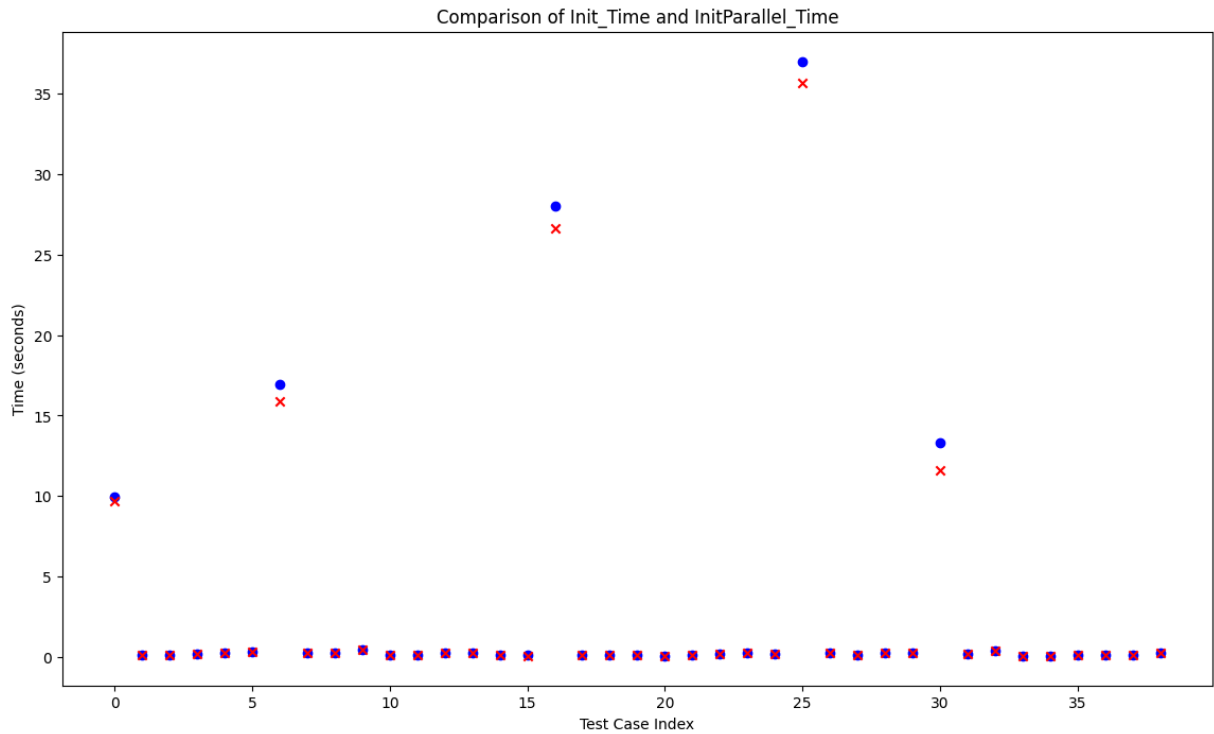
This scatter plot visualizes the relationship between the **initialization time** (`Init_Time`) and **parallel initialization time** (`InitParallel_Time`).

- **X-axis:** Represents the test cases.
- **Y-axis:** Represents the initialization times in seconds.
- **Blue Circles:** Represent `Init_Time` .
- **Red Crosses:** Represent `InitParallel_Time` .

The plot helps to compare the performance of standard initialization versus parallel initialization, showing how closely they relate across different test cases.

```
In [423... plt.figure(figsize=(14, 8))
plt.xlabel('Test Case Index')
plt.ylabel('Time (seconds)')
plt.title('Comparison of Init_Time and InitParallel_Time')
plt.scatter(df_clean_InitParallelWithConcurrentDictionary.index, df_clean_InitParallelTime)
plt.scatter(df_clean_InitParallelWithConcurrentDictionary.index, df_clean_InitParallelTime)
```

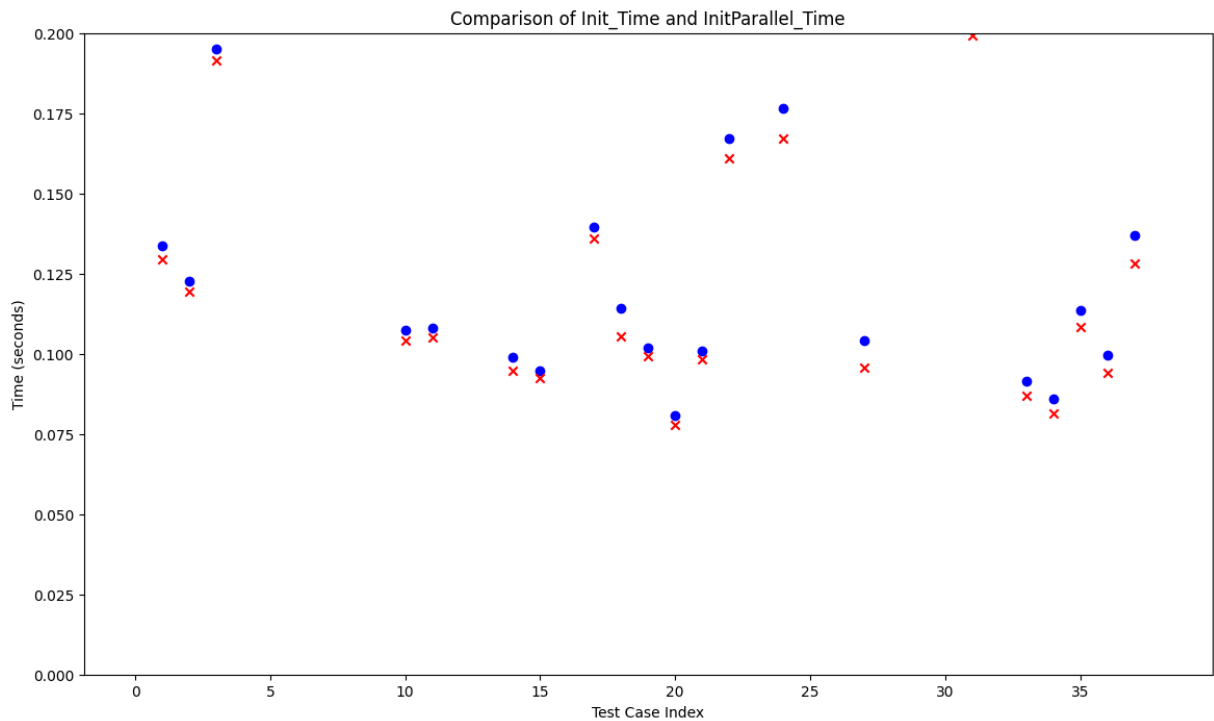
```
Out[423... <matplotlib.collections.PathCollection at 0x20c5b07a8d0>
```



Scatter plot comparing `Init_Time` (blue circles) and `InitParallel_Time` (red crosses) with the y-axis limited to 0-0.2 seconds for better visibility.

```
In [414... plt.figure(figsize=(14, 8))
plt.scatter(df_clean_InitParallelWithConcurrentDictionary.index, df_clean_InitParal
plt.scatter(df_clean_InitParallelWithConcurrentDictionary.index, df_clean_InitParal
plt.ylim(0, 0.2)
plt.xlabel('Test Case Index')
plt.ylabel('Time (seconds)')
plt.title('Comparison of Init_Time and InitParallel_Time')
```

```
Out[414... Text(0.5, 1.0, 'Comparison of Init_Time and InitParallel_Time')
```



Scatter Plot for Compute Time Comparison

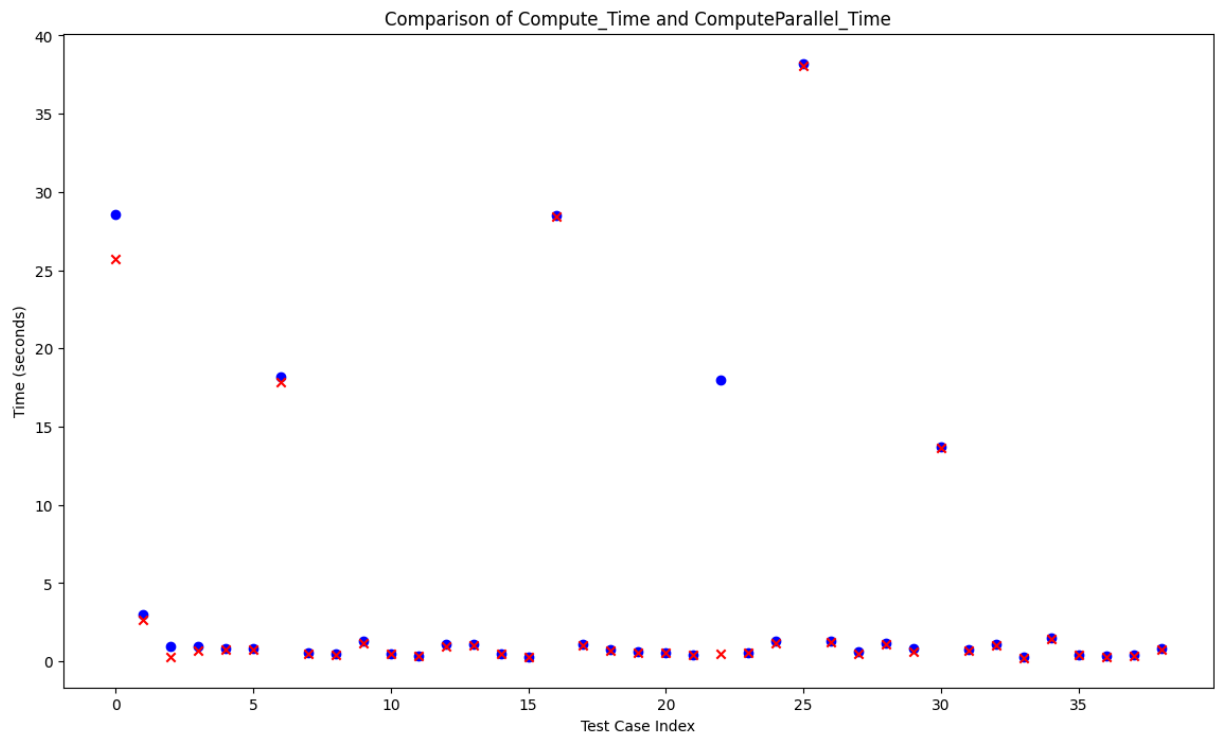
This scatter plot visualizes the relationship between the **Computing time** (`Compute_Time`) and **parallel Computing time** (`ComputeParallel_Time`).

- **X-axis:** Represents the test cases.
- **Y-axis:** Represents the initialization times in seconds.
- **Blue Circles:** Represent `Compute_Time` .
- **Red Crosses:** Represent `ComputeParallel_Time` .

The plot helps to compare the performance of standard initialization versus parallel initialization, showing how closely they relate across different test cases.

```
In [422... plt.figure(figsize=(14, 8))
plt.xlabel('Test Case Index')
plt.ylabel('Time (seconds)')
plt.title('Comparison of Compute_Time and ComputeParallel_Time')
plt.scatter(df_clean_InitParallelWithConcurrentDictionary.index, df_clean_InitParallelTime)
plt.scatter(df_clean_InitParallelWithConcurrentDictionary.index, df_clean_InitParallelTime)
```

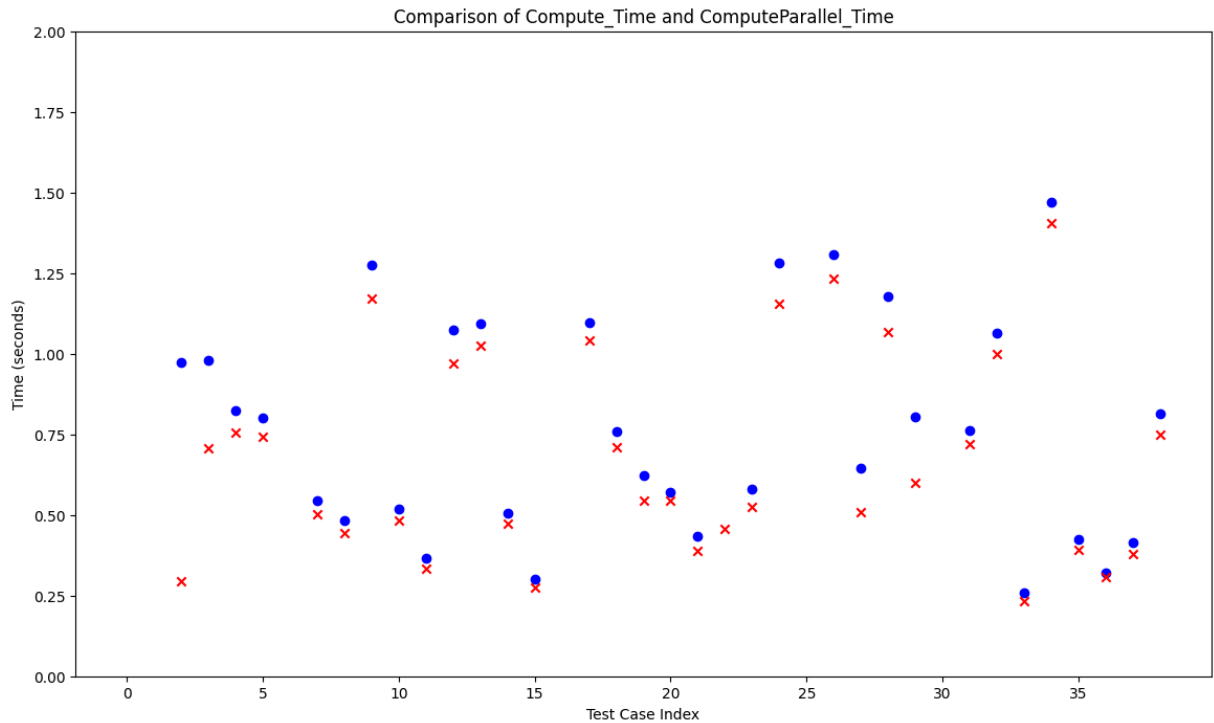
```
Out[422... <matplotlib.collections.PathCollection at 0x20c5d853590>
```

Scatter plot comparing `Compute_Time` (blue circles) and `ComputeParallel_Time` (red crosses) with the y-axis limited to 0-5 seconds for better visibility.

```
In [421... plt.figure(figsize=(14, 8))
plt.scatter(df_clean_InitParallelWithConcurrentDictionary.index, df_clean_InitParal
plt.scatter(df_clean_InitParallelWithConcurrentDictionary.index, df_clean_InitParal
plt.ylim(0, 2)
plt.xlabel('Test Case Index')
plt.ylabel('Time (seconds)')
plt.title('Comparison of Compute_Time and ComputeParallel_Time')
```

```
Out[421... Text(0.5, 1.0, 'Comparison of Compute_Time and ComputeParallel_Time')
```



Scatter Plot for `InitParallel_Time` vs `ComputeParallel_Time`

The following scatter plot visualizes the relationship between `InitParallel_Time` and `ComputeParallel_Time`. The x-axis represents `InitParallel_Time`, and the y-axis represents `ComputeParallel_Time`, both in seconds. This plot helps to observe how changes in initialization times are related to compute times.

```
In [ ]: import plotly.express as px

fig = px.scatter(df_clean_InitParallelWithConcurrentDictionary,
                 x="InitParallel_Time", y="ComputeParallel_Time")

fig.update_layout(
    title="Scatter Plot of InitParallel_Time vs ComputeParallel_Time",
    xaxis_title="InitParallel_Time (Seconds)",
    yaxis_title="ComputeParallel_Time (Seconds)"
)

fig.show()
```

Comparison of New Method (InitParallel - Multi-thread) vs. Old Method (Init - Single-thread) Across Various CPU Cores

In this section, we compare the performance of the new multi-threaded method (`InitParallel`) with the old single-threaded method (`Init`) across different CPU core

configurations (4, 6, 8, 10 and All cores). The results are visualized through box-and-whisker plots, highlighting the differences in initialization and computation times.

- **New Method (InitParallel):** Utilizes a multi-threading approach for faster initialization.
- **Old Method (Init):** Relies on a single-thread approach for initialization.

We analyze the impact of CPU cores on the performance of both methods, focusing on time efficiency across different core configurations.

In [136...

```
csv_file_path_1 = 'D:/Aayush_CPU_4.csv'
csv_file_path_2 = 'D:/Aaditya_CPU_4.csv'
csv_file_path_3 = 'D:/Omkar_CPU_4.csv'

df1 = pd.read_csv(csv_file_path_1)
df2 = pd.read_csv(csv_file_path_2)
df3 = pd.read_csv(csv_file_path_3)

merged_df_CPU_4 = pd.merge(pd.merge(df1, df2, on="Test_Case", how="inner"), df3, on
#display(merged_df_CPU_4)

csv_file_path_4 = 'D:/Aayush_CPU_6.csv'
csv_file_path_5 = 'D:/Aaditya_CPU_6.csv'
csv_file_path_6 = 'D:/Omkar_CPU_6.csv'

df1 = pd.read_csv(csv_file_path_4)
df2 = pd.read_csv(csv_file_path_5)
df3 = pd.read_csv(csv_file_path_6)

merged_df_CPU_6 = pd.merge(pd.merge(df1, df2, on="Test_Case", how="inner"), df3, on
#display(merged_df_CPU_6)

csv_file_path_7 = 'D:/Aayush_CPU_8.csv'
csv_file_path_8 = 'D:/Aaditya_CPU_8.csv'
csv_file_path_9 = 'D:/Omkar_CPU_8.csv'

df1 = pd.read_csv(csv_file_path_7)
df2 = pd.read_csv(csv_file_path_8)
df3 = pd.read_csv(csv_file_path_9)

merged_df_CPU_8 = pd.merge(pd.merge(df1, df2, on="Test_Case", how="inner"), df3, on
#display(merged_df_CPU_8)

csv_file_path_10 = 'D:/Aayush_CPU_10.csv'
csv_file_path_11 = 'D:/Aaditya_CPU_10.csv'
csv_file_path_12 = 'D:/Omkar_CPU_10.csv'

df1 = pd.read_csv(csv_file_path_10)
df2 = pd.read_csv(csv_file_path_11)
df3 = pd.read_csv(csv_file_path_12)

merged_df_CPU_10 = pd.merge(pd.merge(df1, df2, on="Test_Case", how="inner"), df3, on
```

```

csv_file_path_13 = 'D:/Aayush_All_CPU.csv'
csv_file_path_14= 'D:/Aaditya_All_CPU.csv'
csv_file_path_15= 'D:/Omkar_All_CPU.csv'

df1 = pd.read_csv(csv_file_path_13)
df2 = pd.read_csv(csv_file_path_14)
df3 = pd.read_csv(csv_file_path_15)

merged_df_all= pd.merge(pd.merge(df1, df2, on="Test_Case", how="inner"), df3, on="T

#display(merged_df_CPU_10)

```

Comparing for All CPU Core

Graph between InitParallel_Time vs ComputeParallel_Time for All CPU core on differnt pcs

In [135...

```

import matplotlib.pyplot as plt

# Data for the box plots (you already have this)
data_1 = [
    merged_df_CPU_4["InitParallel_Time_x"].dropna(),
    merged_df_CPU_4["InitParallel_Time_y"].dropna(),
    merged_df_CPU_4["InitParallel_Time"].dropna(),
    merged_df_CPU_4["ComputeParallel_Time_x"].dropna(),
    merged_df_CPU_4["ComputeParallel_Time_y"].dropna(),
    merged_df_CPU_4["ComputeParallel_Time"].dropna()
]

data_2 = [
    merged_df_CPU_6["InitParallel_Time_x"].dropna(),
    merged_df_CPU_6["InitParallel_Time_y"].dropna(),
    merged_df_CPU_6["InitParallel_Time"].dropna(),
    merged_df_CPU_6["ComputeParallel_Time_x"].dropna(),
    merged_df_CPU_6["ComputeParallel_Time_y"].dropna(),
    merged_df_CPU_6["ComputeParallel_Time"].dropna()
]

data_3 = [
    merged_df_CPU_8["InitParallel_Time_x"].dropna(),
    merged_df_CPU_8["InitParallel_Time_y"].dropna(),
    merged_df_CPU_8["InitParallel_Time"].dropna(),
    merged_df_CPU_8["ComputeParallel_Time_x"].dropna(),
    merged_df_CPU_8["ComputeParallel_Time_y"].dropna(),
    merged_df_CPU_8["ComputeParallel_Time"].dropna()
]

data_4 = [
    merged_df_CPU_10["InitParallel_Time_x"].dropna(),
    merged_df_CPU_10["InitParallel_Time_y"].dropna(),

```

```

merged_df_CPU_10["InitParallel_Time"].dropna(),
merged_df_CPU_10["ComputeParallel_Time_x"].dropna(),
merged_df_CPU_10["ComputeParallel_Time_y"].dropna(),
merged_df_CPU_10["ComputeParallel_Time"].dropna()
]

data_5 = [
merged_df_all["InitParallel_Time_x"].dropna(),
merged_df_all["InitParallel_Time_y"].dropna(),
merged_df_all["InitParallel_Time"].dropna(),
merged_df_all["ComputeParallel_Time_x"].dropna(),
merged_df_all["ComputeParallel_Time_y"].dropna(),
merged_df_all["ComputeParallel_Time"].dropna()
]

# Labels for each box plot
labels = ["Init_Time(Member_1)", "Init_Time(Member_2)", "Init_Time(Member_3)", "Com

# Colors for each box
colors = ["blue", "orange", "red", "gray", "yellow", "green"]

# Create a figure with five subplots (1 column and 5 rows)
fig, axes = plt.subplots(5, 1, figsize=(14, 20))

# First plot (Box and Whisker Plot for data_1)
box_1 = axes[0].boxplot(data_1, labels=labels, patch_artist=True)
for patch, color in zip(box_1["boxes"], colors):
    patch.set_facecolor(color)
axes[0].set_ylim(0, 2) # Set Y-axis range
axes[0].axvline(x=3.5, color='black', linestyle='--', linewidth=2)
axes[0].set_ylabel("Time (ms)")
axes[0].set_title("    Box and Whisker Plot of InitParallel_Time(CPU_Core=4)

# Second plot (Box and Whisker Plot for data_2)
box_2 = axes[1].boxplot(data_2, labels=labels, patch_artist=True)
for patch, color in zip(box_2["boxes"], colors):
    patch.set_facecolor(color)
axes[1].set_ylim(0, 2)
axes[1].axvline(x=3.5, color='black', linestyle='--', linewidth=2)
axes[1].set_ylabel("Time (ms)")
axes[1].set_title("    Box and Whisker Plot of InitParallel_Time(CPU_Core=6)

# Third plot (Box and Whisker Plot for data_3)
box_3 = axes[2].boxplot(data_3, labels=labels, patch_artist=True)
for patch, color in zip(box_3["boxes"], colors):
    patch.set_facecolor(color)
axes[2].set_ylim(0, 2)
axes[2].axvline(x=3.5, color='black', linestyle='--', linewidth=2)
axes[2].set_ylabel("Time (ms)")
axes[2].set_title("    Box and Whisker Plot of InitParallel_Time(CPU_Core=8)

# Fourth plot (Box and Whisker Plot for data_4)
box_4 = axes[3].boxplot(data_4, labels=labels, patch_artist=True)

```

```

for patch, color in zip(box_4["boxes"], colors):
    patch.set_facecolor(color)
axes[3].set_ylim(0, 2)
axes[3].axvline(x=3.5, color='black', linestyle='--', linewidth=2)
axes[3].set_ylabel("Time (ms)")
axes[3].set_title("    Box and Whisker Plot of InitParallel_Time(CPU_Core=10)

# Fifth plot (Box and Whisker Plot for data_5)
box_5 = axes[4].boxplot(data_5, labels=labels, patch_artist=True)
for patch, color in zip(box_5["boxes"], colors):
    patch.set_facecolor(color)
axes[4].set_ylim(0, 2)
axes[4].axvline(x=3.5, color='black', linestyle='--', linewidth=2)
axes[4].set_ylabel("Time (ms)")
axes[4].set_title("    Box and Whisker Plot of InitParallel_Time(CPU_Core=All)

# Adjust layout to avoid overlap
plt.tight_layout()

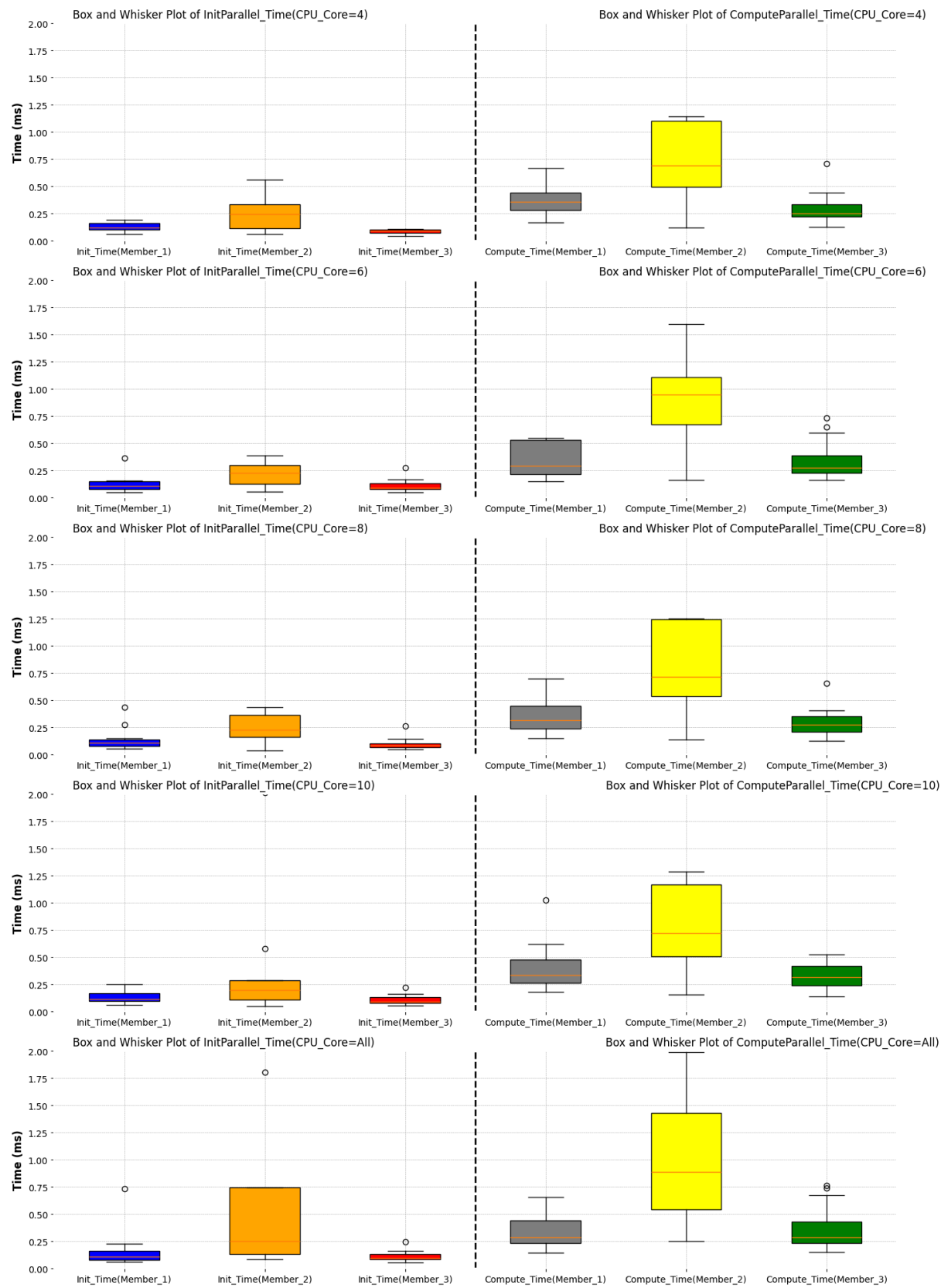
# Show the plot
plt.show()

```

```

C:\Users\Omkar\AppData\Local\Temp\ipykernel_10612\3070730397.py:59: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
    box_1 = axes[0].boxplot(data_1, labels=labels, patch_artist=True)
C:\Users\Omkar\AppData\Local\Temp\ipykernel_10612\3070730397.py:69: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
    box_2 = axes[1].boxplot(data_2, labels=labels, patch_artist=True)
C:\Users\Omkar\AppData\Local\Temp\ipykernel_10612\3070730397.py:79: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
    box_3 = axes[2].boxplot(data_3, labels=labels, patch_artist=True)
C:\Users\Omkar\AppData\Local\Temp\ipykernel_10612\3070730397.py:89: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
    box_4 = axes[3].boxplot(data_4, labels=labels, patch_artist=True)
C:\Users\Omkar\AppData\Local\Temp\ipykernel_10612\3070730397.py:99: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
    box_5 = axes[4].boxplot(data_5, labels=labels, patch_artist=True)

```



Comparing the Performance of the Newly Created Multi-Thread Method

Evaluation Across Different PCs Using Various CPU Cores

In [109...

```

import matplotlib.pyplot as plt

# Data for the box plots (you already have this)
data_1 = [
    merged_df_CPU_4["InitParallel_Time_x"].dropna(),
    merged_df_CPU_4["InitParallel_Time_y"].dropna(),
    merged_df_CPU_4["InitParallel_Time"].dropna(),
    merged_df_CPU_4["Init_Time_x"].dropna(),
    merged_df_CPU_4["Init_Time_y"].dropna(),
    merged_df_CPU_4["Init_Time"].dropna()
]

data_2 = [
    merged_df_CPU_6["InitParallel_Time_x"].dropna(),
    merged_df_CPU_6["InitParallel_Time_y"].dropna(),
    merged_df_CPU_6["InitParallel_Time"].dropna(),
    merged_df_CPU_6["Init_Time_x"].dropna(),
    merged_df_CPU_6["Init_Time_y"].dropna(),
    merged_df_CPU_6["Init_Time"].dropna()
]

data_3 = [
    merged_df_CPU_8["InitParallel_Time_x"].dropna(),
    merged_df_CPU_8["InitParallel_Time_y"].dropna(),
    merged_df_CPU_8["InitParallel_Time"].dropna(),
    merged_df_CPU_8["Init_Time_x"].dropna(),
    merged_df_CPU_8["Init_Time_y"].dropna(),
    merged_df_CPU_8["Init_Time"].dropna()
]

data_4 = [
    merged_df_CPU_10["InitParallel_Time_x"].dropna(),
    merged_df_CPU_10["InitParallel_Time_y"].dropna(),
    merged_df_CPU_10["InitParallel_Time"].dropna(),
    merged_df_CPU_10["Init_Time_x"].dropna(),
    merged_df_CPU_10["Init_Time_y"].dropna(),
    merged_df_CPU_10["Init_Time"].dropna()
]

data_5 = [
    merged_df_all["InitParallel_Time_x"].dropna(),
    merged_df_all["InitParallel_Time_y"].dropna(),
    merged_df_all["InitParallel_Time"].dropna(),
    merged_df_all["Init_Time_x"].dropna(),
    merged_df_all["Init_Time_y"].dropna(),
    merged_df_all["Init_Time"].dropna()
]

# Labels for each box plot
labels = ["InitParallel_Time(Member_1)", "InitParallel_Time(Member_2)", "InitParallel_Time(Member_3)",
          "Init_Time(Member_1)", "Init_Time(Member_2)", "Init_Time(Member_3)"]

# Colors for each box
colors = ["blue", "orange", "red", "gray", "yellow", "green"]

```



```

# Create a figure with five subplots (1 column and 5 rows)
fig, axes = plt.subplots(5, 1, figsize=(14, 20))

# First plot (Box and Whisker Plot for data_1)
box_1 = axes[0].boxplot(data_1, labels=labels, patch_artist=True)
for patch, color in zip(box_1["boxes"], colors):
    patch.set_facecolor(color)
axes[0].set_ylim(0, 0.75) # Set Y-axis range
axes[0].axvline(x=3.5, color='black', linestyle='--', linewidth=2)
axes[0].set_ylabel("Time (ms)")
axes[0].set_title("Comparison Between New Method (InitParallel - Multi-thread) and

# Second plot (Box and Whisker Plot for data_2)
box_2 = axes[1].boxplot(data_2, labels=labels, patch_artist=True)
for patch, color in zip(box_2["boxes"], colors):
    patch.set_facecolor(color)
axes[1].set_ylim(0, 0.75)
axes[1].axvline(x=3.5, color='black', linestyle='--', linewidth=2)
axes[1].set_ylabel("Time (ms)")
axes[1].set_title("Comparison Between New Method (InitParallel - Multi-thread) and

# Third plot (Box and Whisker Plot for data_3)
box_3 = axes[2].boxplot(data_3, labels=labels, patch_artist=True)
for patch, color in zip(box_3["boxes"], colors):
    patch.set_facecolor(color)
axes[2].set_ylim(0, 0.75)
axes[2].axvline(x=3.5, color='black', linestyle='--', linewidth=2)
axes[2].set_ylabel("Time (ms)")
axes[2].set_title("Comparison Between New Method (InitParallel - Multi-thread) and

# Fourth plot (Box and Whisker Plot for data_4)
box_4 = axes[3].boxplot(data_4, labels=labels, patch_artist=True)
for patch, color in zip(box_4["boxes"], colors):
    patch.set_facecolor(color)
axes[3].set_ylim(0, 0.75)
axes[3].axvline(x=3.5, color='black', linestyle='--', linewidth=2)
axes[3].set_ylabel("Time (ms)")
axes[3].set_title("Comparison Between New Method (InitParallel - Multi-thread) and

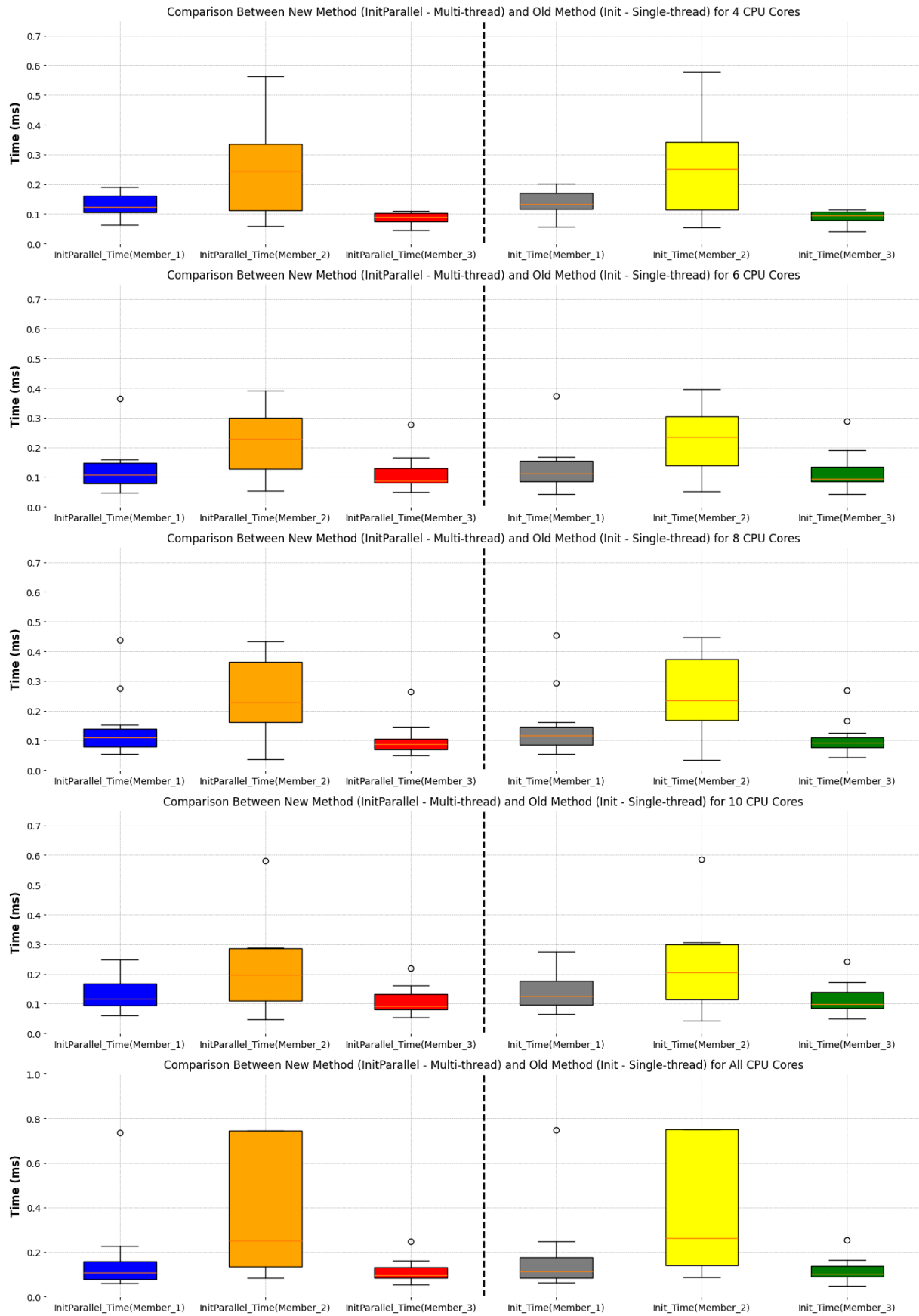
# Fifth plot (Box and Whisker Plot for data_5)
box_5 = axes[4].boxplot(data_5, labels=labels, patch_artist=True)
for patch, color in zip(box_5["boxes"], colors):
    patch.set_facecolor(color)
axes[4].set_ylim(0, 1)
axes[4].axvline(x=3.5, color='black', linestyle='--', linewidth=2)
axes[4].set_ylabel("Time (ms)")
axes[4].set_title("Comparison Between New Method (InitParallel - Multi-thread) and

# Adjust layout to avoid overlap
plt.tight_layout()

# Show the plot
plt.show()

```

```
C:\Users\Omkar\AppData\Local\Temp\ipykernel_10612\3883627221.py:60: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
    box_1 = axes[0].boxplot(data_1, labels=labels, patch_artist=True)
C:\Users\Omkar\AppData\Local\Temp\ipykernel_10612\3883627221.py:70: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
    box_2 = axes[1].boxplot(data_2, labels=labels, patch_artist=True)
C:\Users\Omkar\AppData\Local\Temp\ipykernel_10612\3883627221.py:79: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
    box_3 = axes[2].boxplot(data_3, labels=labels, patch_artist=True)
C:\Users\Omkar\AppData\Local\Temp\ipykernel_10612\3883627221.py:88: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
    box_4 = axes[3].boxplot(data_4, labels=labels, patch_artist=True)
C:\Users\Omkar\AppData\Local\Temp\ipykernel_10612\3883627221.py:97: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
    box_5 = axes[4].boxplot(data_5, labels=labels, patch_artist=True)
```



The End !!!